Pashov Audit Group

Saffron Security Review

March 6th 2026 - March 16th 2026

Contents

1. About Pashov Audit Group ... 3
2. Disclaimer ... 3
3. Risk Classification ... 3
4. About Saffron Orca ... 4
5. Executive Summary ... 4
6. Findings ... 5

High findings ... 7

[H-01] `deposit_fixed` does not validate tick range against vault's `tick_lower` and `tick_upper` ... 7
[H-02] Fixed deposit allows unauthorized premium capture and tolerance bypass ... 8

Medium findings ... 10

[M-01] Hardcoded `remaining_accounts_info` prevent vault fund transfers ... 10
[M-02] User `withdraw_variable` can be called after admin_fixed_withdraw is set ... 12
[M-03] Early_withdraw_fixed incorrectly clears `fixed_side_capacity` and tick value ... 13
[M-04] Admin closure relies on outdated depositor address in vault ... 14
[M-05] Unvalidated token mints in `create_vault` lead to withdrawal failure ... 16

Low findings ... 19

[L-01] Admin close cannot complete after early_exit_fixed ... 19
[L-02] Acceptance of unverified pools in Saffron from Whirlpool program ... 19
[L-03] `initialize_factory` allows unauthorized address to gain authority ... 20
[L-04] Frozen depositor ATA permanently blocks admin vault close ... 21
[L-05] Missing anchor discriminator validation on manually parsed Orca account ... 22
[L-06] `VaultState::get_state()` off-by-one at `end_time` boundary ... 23
[L-07] `admin_close_vault` does not close vault ATAs rent locked ... 23
[L-08] Orca incentive rewards are not attributed ... 24
[L-09] Withdraw handlers do not check for admin-withdrawn flags ... 24
[L-10] Claim leaves user `claim` token ATA open after burn rent not returned ... 24
[L-11] Truncation of `remaining_premium` undercharges early exit penalty ... 25
[L-12] Tick bounds in Saffron do not match Orca's canonical range ... 25
[L-13] Variable asset ATA created in `deposit_variable` instead of `initialize_vault` ... 26
[L-14] Front-running on `create_vault` ... 27
[L-15] Missing ownership verification in `withdraw.rs` allows unauthorized withdrawal ... 27
[L-16] Unclaimed premium lost to fixed depositor if `claim` is not called in time ... 28
[L-17] Pre-vault-start fees incorrectly assigned to variable side instead of fixed side ... 29
[L-18] Overaccounting early-exit penalty leads to insufficient vault balance ... 31

1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over $100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work here or reach out on Twitter @pashovkrum.

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive

4. About Saffron Orca

Saffron Orca is a Solana-based fixed-income vault protocol that allows users to split yield from Orca liquidity positions into fixed and variable tranches. The vault program manages deposits, fee accrual, withdrawal flows, and position lifecycle through a set of on-chain instructions built with Anchor.

5. Executive Summary

A time-boxed security review of the **saffron-finance/orca-saffron** repository was done by Pashov Audit Group, during which **0x15, ZeroTrust01, 0xAlix2, newspace, shaflow** engaged to review **Saffron Orca**. A total of **25** issues were uncovered.

Protocol Summary	
Project Name	Saffron Orca
Protocol Type	Fixed-income vault on Orca
Timeline	March 6th 2026 - March 16th 2026

Review commit hash:
- 56b254ac8c19a640318e40b8b40e0240b36e5c9f (saffron-finance/orca-saffron)

Fixes review commit hash:
- 09338a17274492d0bcb60233b2a5e6429fdfd75c (saffron-finance/orca-saffron)

Scope

`admin.rs` `claim.rs` `constants.rs` `create.rs` `deposit_fixed.rs` `deposit_variable.rs` `early_exit.rs` `early_withdraw.rs` `error.rs` `events.rs` `factory.rs` `initialize.rs` `lib.rs` `math.rs` `mod.rs` `tick_arrays.rs` `token.rs` `transfer_authority.rs` `vault.rs` `withdraw.rs`

6. Findings

Findings count

Severity	Amount
High	2
Medium	5
Low	18
Total findings	**25**

Summary of findings

ID	Title	Severity	Status
[H-01]	`deposit_fixed` does not validate tick range against vault's `tick_lower` and `tick_upper`	High	Resolved
[H-02]	Fixed deposit allows unauthorized premium capture and tolerance bypass	High	Resolved
[M-01]	Hardcoded `remaining_accounts_info` prevent vault fund transfers	Medium	Resolved
[M-02]	User `withdraw_variable` can be called after admin_fixed_withdraw is set	Medium	Resolved
[M-03]	Early_withdraw_fixed incorrectly clears `fixed_side_capacity` and tick value	Medium	Resolved
[M-04]	Admin closure relies on outdated depositor address in vault	Medium	Resolved
[M-05]	Unvalidated token mints in `create_vault` lead to withdrawal failure	Medium	Resolved
[L-01]	Admin close cannot complete after early_exit_fixed	Low	Resolved
[L-02]	Acceptance of unverified pools in Saffron from Whirlpool program	Low	Resolved
[L-03]	`initialize_factory` allows unauthorized address to gain authority	Low	Resolved
[L-04]	Frozen depositor ATA permanently blocks admin vault close	Low	Acknowledged

ID	Title	Severity	Status
[L-05]	Missing anchor discriminator validation on manually parsed Orca account	Low	Resolved
[L-06]	`VaultState::get_state()` off-by-one at `end_time` boundary	Low	Resolved
[L-07]	`admin_close_vault` does not close vault ATAs rent locked	Low	Resolved
[L-08]	Orca incentive rewards are not attributed	Low	Acknowledged
[L-09]	Withdraw handlers do not check for admin-withdrawn flags	Low	Resolved
[L-10]	Claim leaves user `claim` token ATA open after burn rent not returned	Low	Resolved
[L-11]	Truncation of `remaining_premium` undercharges early exit penalty	Low	Resolved
[L-12]	Tick bounds in Saffron do not match Orca's canonical range	Low	Resolved
[L-13]	Variable asset ATA created in `deposit_variable` instead of `initialize_vault`	Low	Resolved
[L-14]	Front-running on `create_vault`	Low	Resolved
[L-15]	Missing ownership verification in `withdraw.rs` allows unauthorized withdrawal	Low	Resolved
[L-16]	Unclaimed premium lost to fixed depositor if `claim` is not called in time	Low	Resolved
[L-17]	Pre-vault-start fees incorrectly assigned to variable side instead of fixed side	Low	Resolved
[L-18]	Overaccounting early-exit penalty leads to insufficient vault balance	Low	Resolved

High findings

[H-01] `deposit_fixed` does not validate tick range against vault's `tick_lower` and `tick_upper`

Severity

Impact: High

Likelihood: Medium

Description

The vault is created with `tick_lower` and `tick_upper` at `create_vault`; these are validated (alignment, range) and stored on the vault and emitted in `VaultCreated`. They define the concentration range for which the variable side pays a premium and expects a corresponding level of fee generation.

In concentrated liquidity, for a given liquidity amount L, a **narrower** tick range yields **higher** fee generation when the price is in range; a **wider** range spreads L over more price levels and yields **lower** fees per unit volume. So a vault created for ticks T1 and T2 with liquidity L implies an expected fee level X; if the actual position has the same L but a much wider range [T1′, T2′], fee generation will be **less** than X.

In `deposit_fixed`, the position's `tick_lower_index` and `tick_upper_index` are parsed from the Orca position account. The only tick-related logic is to **overwrite** the vault's stored ticks with the position's values. There is **no check** that the position's tick range matches the vault's `tick_lower` and `tick_upper`.

```
// Update vault state with position data
{
    let mut vault = ctx.accounts.vault.load_mut()?;
    vault.tick_lower = position_data.tick_lower_index;
    vault.tick_upper = position_data.tick_upper_index;
    vault.set_position_mint(Some(ctx.accounts.position_mint.key()));
    ...
    vault.fixed_side_capacity = position_data.liquidity;
    ...
}
```

A fixed depositor can therefore deposit a position with a **different** (e.g., much wider) tick range than the vault was created for.

The variable side, having paid a premium based on the created range and capacity, receives fee rights over a position that generates **less** fee income than expected.

Attack scenario: (1) Vault creator advertises a tight, in-range tick spread implying high expected fee yield. (2) Variable depositor pays full premium based on the advertised range. (3) Fixed depositor deposits a position with a completely different tick range — for example, far out of the current price — that generates zero fees. (4) Vault starts; variable depositor is locked in for the full duration with no recourse. This constitutes direct theft of the variable depositor's premium via misrepresented vault configuration.

Recommendations

Consider validating the position's tick range against the vault's before accepting the deposit. Require that `position_data.tick_lower_index == vault.tick_lower` and `position_data.tick_upper_index == vault.tick_upper`, so that the deposited position matches the range for which the vault was created, and the variable side receives the fee level implied by the creation parameters.

[H-02] Fixed deposit allows unauthorized premium capture and tolerance bypass

Severity

Impact: High

Likelihood: Medium

Description

The vault is initialized with fixed-side economic commitments, but `deposit_fixed` does not enforce those commitments. A fixed depositor can submit any same-Whirlpool position with `liquidity > 0`, overwrite vault terms, trigger start, and then claim the full premium.

The unused tolerance field also creates a false sense of protection for integrators, frontends, and off-chain pricing logic, since the contract stores a tolerance value but never enforces it in the production deposit path.

The existing `validate_liquidity_tolerance()` helper enforces two bounds: `actual_liquidity > expected * (MAX_BPS - deposit_tolerance_bps) / MAX_BPS` (reject if too low) and `actual_liquidity <= expected * (MAX_BPS + deposit_tolerance_bps) / MAX_BPS` (reject if too high). Neither check is invoked in `deposit_fixed`.

Code references

- Fixed-side commitment is set at initialization: initialize.rs
- Tolerance configuration is stored at vault creation: create.rs
- Tolerance validator exists but is not used in `deposit_fixed`: vault.rs
- Variable side must deposit full premium in one shot: deposit_variable.rs
- `deposit_fixed` only checks whirlpool/mint/liquidity > 0: deposit_fixed.rs, deposit_fixed.rs
- `deposit_fixed` overwrites ticks and fixed capacity from attacker position: deposit_fixed.rs, deposit_fixed.rs
- Vault auto-starts when both NFTs exist (not economic quality): deposit_fixed.rs
- Claim is proportional to claim-token share; with supply = 1 holder gets all premium: claim.rs, claim.rs

Root cause

`fixed_side_capacity` and tolerance are treated as "initial values" but are not enforced at the economically binding step (`deposit_fixed`). `deposit_fixed` mutates the vault to the deposited position instead of validating the deposited position against prior commitments.

Attack path

1. Victim deposits full variable premium (`deposit_variable` requires full capacity).
2. Attacker deposits a dust/low-liquidity position from the same Whirlpool.
3. `deposit_fixed` accepts it, overwrites `fixed_side_capacity`, mints the sole claim token, and starts the vault.
4. Attacker calls `claim`; since claim supply is 1, the attacker receives the entire vault premium.
5. Attacker later exits/withdraws position value separately.

Impact

- Direct premium extraction from variable side (theft/extraction).
- Economic integrity break: advertised fixed terms are not binding.
- Per-vault loss bounded by `variable_side_capacity`.
- Repeatable across eligible vaults.

Recommendation

1. Enforce immutable fixed commitments in `deposit_fixed`.
2. Require exact tick match with committed ticks (`tick_lower`, `tick_upper`) instead of overwriting.
3. Keep `fixed_side_capacity` immutable after initialization, and validate deposited liquidity via `validate_liquidity_tolerance(expected, actual)` before minting claim token.

Medium findings

[M-01] Hardcoded `remaining_accounts_info` prevent vault fund transfers

Severity

Impact: Medium

Likelihood: Medium

Description

Saffron Vault manually constructs CPI instructions to call Orca Whirlpool's V2 instructions (`collect_fees_v2`, `increase_liquidity_v2`, `decrease_liquidity_v2`). In the CPI wrappers, the `remaining_accounts_info` parameter is hardcoded to `None` (i.e. `data.push(0)`), and no `remaining_accounts` are passed:

```
// programs/saffron-vault/src/cpi/orca.rs L757-759
let mut data = Vec::with_capacity(8 + 1);
data.extend_from_slice(&COLLECT_FEES_V2_DISCRIMINATOR);
data.push(0); // remaining_accounts_info: None
```

Orca Whirlpool's V2 instructions support the Token-2022 Transfer Hook extension through the `remaining_accounts` mechanism. When a token mint has a transfer hook enabled, Whirlpool extracts the extra accounts required by the transfer hook from `remaining_accounts` before executing token transfers:

```
// whirlpools/programs/whirlpool/src/util/v2/token.rs L151-154
if let Some(hook_program_id) = get_transfer_hook_program_id(token_mint)? {
    let transfer_hook_accounts = transfer_hook_accounts
        .as_deref()
        .ok_or(ErrorCode::NoExtraAccountsForTransferHook)?; // ← reverts here
    // ...
}
```

Whirlpool **does allow** creating pools with Token-2022 tokens that have Transfer Hook enabled (as long as the token has a TokenBadge). This is explicitly reflected in `is_supported_token_mint`:

```
// whirlpools/programs/whirlpool/src/util/v2/token.rs L266-270
TokenExtensionType::TransferHook => {
    if !is_token_badge_initialized {
        return Ok(false);
    }
}
```

Therefore, when a Saffron Vault is connected to a Whirlpool that contains a Transfer Hook token:

1. The `increase_liquidity_v2` CPI in `deposit_fixed` will fail → unable to deposit liquidity.
2. If the vault already holds a position by other means (e.g., the token enables its transfer hook after deposit), then:
3. The `collect_fees_v2` CPI in `withdraw_fixed` will fail.
4. The `collect_fees_v2` CPI in `early_exit_fixed` will fail.
5. The `collect_fees_v2` CPI in `admin_withdraw_fixed` will fail.
6. `decrease_liquidity_v2` will also fail.

All exit paths are blocked, and the liquidity position and accrued fees in the vault become permanently unrecoverable.

Affected CPI call sites:

The table outlines the specific CPI (Cross-Program Invocation) functions that are affected by the hardcoding of the `remaining_accounts_info` parameter as `None`, which leads to the permanent locking of vault funds for Transfer Hook token pools.

The first function listed is `collect_fees_v2`, which is called from three different sites. The first call site is `withdraw_fixed`, located in the file `instructions/vault/withdraw.rs` at line 531. The second call site is `early_exit_fixed`, found in `instructions/vault/early_exit.rs` at line 267. The third call site for `collect_fees_v2` is `admin_withdraw_fixed`, which can be found in the file `instructions/factory/admin.rs` at line 308.

Additionally, the function `increase_liquidity_v2` is called from the site `deposit_fixed`, which is located in the file `cpi/orca.rs` at line 625. Lastly, the function `decrease_liquidity_v2` is associated with related exit paths, as indicated in the file `cpi/orca.rs` at line 688.

This information highlights the critical points in the code where the hardcoded `remaining_accounts_info` could impact the functionality of the vault, emphasizing the potential risks associated with these specific CPI calls.

Recommendations

Modify the V2 CPI wrappers to support passing `remaining_accounts`:

1. Add optional transfer hook account fields to the `CollectFeesV2Accounts`, `IncreaseLiquidityV2Accounts`, and `DecreaseLiquidityV2Accounts` structs.
2. When constructing the CPI:
3. Properly serialize `remaining_accounts_info` instead of hardcoding it as `None`.
4. Append the extra accounts required by the transfer hook to the `AccountMeta` list and `account_infos`.
5. On the caller side (`withdraw_fixed`, `early_exit_fixed`, etc.), accept `remaining_accounts` and pass them through to the CPI wrapper.

Alternatively, if Saffron does not intend to support Transfer Hook tokens, validate at `create_vault` time whether the Whirlpool's `token_mint_a` and `token_mint_b` use the Transfer Hook extension, and reject vault creation if they do not.

[M-02] User `withdraw_variable` can be called after admin_fixed_withdraw is set

Severity

Impact: Medium

Likelihood: Medium

Description

When the admin has already run `admin_withdraw_fixed` (step 1 of admin close), the vault has no position (`has_position()` is false) and `admin_fixed_withdrawn == 1`. The user-facing `withdraw_variable` handler does not check `admin_fixed_withdrawn`. So a variable depositor can still call `withdraw_variable`, trigger settlement, receive 100% of net variable-side earnings, and burn their bearer token:

`variable_handler` has no check on `admin_fixed_withdrawn`. It only requires vault ended and `!vault.has_position()` (which is true after admin withdraws fixed):

```
// Check vault has ended
require!(
    vault.is_started != 0 && clock.unix_timestamp >= vault.end_time,
    SaffronError::WithdrawBeforeEnd
);

// Fixed side must withdraw first to collect fees before variable side can withdraw.
require!(!vault.has_position(), SaffronError::FixedMustWithdrawFirst);
// ... no require!(vault.admin_fixed_withdrawn == 0, ...)
```

`AdminWithdrawVariable` requires the admin to have withdrawn fixed first; it then takes `ADMIN_CLOSE_FEE_BPS` from earnings and sends the rest to the variable depositor:

```
constraint = vault.load()?.admin_fixed_withdrawn != 0 @ SaffronError::AdminFixedNotWithdrawn,
```

So when the admin later runs `admin_withdraw_variable`, vault earnings are already zero, so the protocol never collects `ADMIN_CLOSE_FEE_BPS` (0.5% of those earnings). That fee is effectively transferred to the variable depositor instead of the fee receiver.

Intended flow: admin withdraws fixed → admin withdraws variable (taking `ADMIN_CLOSE_FEE_BPS` from earnings to fee receiver). Actual flow when user calls `withdraw_variable` in between: user takes full net earnings; admin's later `admin_withdraw_variable` has nothing left to apply the fee on.

Recommendations

In `withdraw_variable` (variable_handler), right after PDA validation, consider adding:

```
require!(vault.admin_fixed_withdrawn == 0, SaffronError::AdminCloseInProgress);
```

(or a dedicated error). Once the admin has withdrawn the fixed side, only `admin_withdraw_variable` should be allowed to move variable-side earnings, so the protocol can collect `ADMIN_CLOSE_FEE_BPS`.

Or acknowledging this behavior, if this loss of fees is acceptable.

[M-03] Early_withdraw_fixed incorrectly clears `fixed_side_capacity` and tick value

Severity

Impact: Medium

Likelihood: Medium

Description

`early_withdraw_fixed` clears all of the following vault fields in its cleanup block:

```
//early_withdraw.rs

    // 3. Clear vault position state (all fields set by deposit_fixed)
    {
        let mut vault = ctx.accounts.vault.load_mut()?;
        vault.set_position_mint(None);
        vault.liquidity = 0;
        vault.fixed_side_capacity = 0;
        vault.fixed_depositor = Pubkey::default();
```

```
        vault.tick_lower = 0;
        vault.tick_upper = 0;
    }
```

However, `fixed_side_capacity`, `tick_lower`, and `tick_upper` were not set by `deposit_fixed` — they are configuration values set at vault creation and initialization. After an early withdrawal, the vault remains open for re-deposit. Clearing these configuration fields means the vault has lost its reference values and re-deposit is impossible. Additionally, if `deposit_fixed` is later updated to call `validate_liquidity_tolerance`, a vault that has had `early_withdraw_fixed` run would have `fixed_side_capacity` equal to 0, making the tolerance check wrong or ineffective for any subsequent re-deposit. The zeroing of these configuration fields creates a latent hazard for future validation logic.

Recommendations

Only clear the fields that `deposit_fixed` wrote and preserve the configuration fields set by `create_vault` and `initialize_vault`.

[M-04] Admin closure relies on outdated depositor address in vault

Severity

Impact: Medium

Likelihood: Medium

Description

1. Deposits snapshot the depositor address into vault state: deposit_fixed.rs:275, deposit_variable.rs:172.
2. Admin close enforces those snapshot addresses, not the current bearer holder: admin.rs:232, admin.rs:453.
3. Admin close routes assets to snapshot accounts: admin.rs:345, admin.rs:627, admin.rs:638, admin.rs:649.
4. User withdraw path is bearer-based (expected semantics): withdraw.rs:188, withdraw.rs:367, and burns the bearer on withdrawal withdraw.rs:314, withdraw.rs:572.
5. Admin path explicitly leaves bearer tokens outstanding: admin.rs:350, admin.rs:690.
6. Intended guard exists but is unused: error.rs:191, with no corresponding check in admin-close constraints admin.rs:731.

Attack Path

1. Attacker deposits fixed/variable and becomes `fixed_depositor` / `variable_depositor`.
2. Attacker transfers/sells bearer rights.
3. Vault reaches end.
4. Admin uses normal admin close flow.
5. Current bearer holder cannot be used as admin payout identity (the proof of concept rejects buyer on admin path).
6. Original depositor receives assets, buyer retains economically void bearer (also demonstrated in proof of concept).

Impact

1. Fixed leg: original depositor can reclaim position NFT after selling fixed bearer rights.
2. Variable leg: original depositor can receive earnings/penalty after selling variable bearer NFT.
3. Breaks bearer-token entitlement invariant and enables value extraction from buyers.
4. Repeatable per vault, constrained mainly by market liquidity and administrative closure occurrence.

POC

```
diff --git a/tests/integration/zz-admin-close-snapshot-routing-poc.test.ts b/tests/
integration/zz-admin-close-snapshot-routing-poc.test.ts
new file mode 100644
index 0000000..c3a4e69

--- /dev/null
+++ b/tests/integration/zz-admin-close-snapshot-routing-poc.test.ts
@@ -0,0 +1,557 @@
+/**

+ * PoC: admin close uses depositor snapshots instead of current bearer ownership.
+ *
+ * This test demonstrates:
+ * 1) bearer rights are transferable (fixed + variable)
+ * 2) admin close path rejects current bearer holder as payout recipient
+ * 3) admin close path succeeds with original depositor snapshots
+ * 4) bearer tokens can remain outstanding while value is routed elsewhere
+ */
```

Recommendations

1. Make entitlement source consistent: admin withdraw must validate and route to current bearer holder, not depositor snapshot.
2. For fixed admin withdraw, require a bearer-holder ATA proof (`mint == fixed_bearer_mint` , `amount > 0`) and route position to that holder's ATA.
3. For variable admin withdraw, require variable bearer-holder ATA proof (`mint == variable_bearer_mint` , `amount > 0`) and route earnings/penalty to that holder. Alternatively, if the protocol does not intend bearer tokens to be transferable, apply a freeze authority or add transfer restrictions to the bearer token mints at initialization time, preventing secondary transfers entirely.

[M-05] Unvalidated token mints in `create_vault` lead to withdrawal failure

Severity

Impact: Medium

Likelihood: Medium

Description

`create_vault` stores caller-supplied `token_mint_a` / `token_mint_b` without verifying they are the actual mints of the provided Whirlpool. Later instructions trust those stored mints and pass them into Orca CPI calls that do enforce Whirlpool↔mint consistency, causing hard reverts at withdrawal time.

Code References

- `create_vault` accepts arbitrary mint accounts and only validates Whirlpool owner + tick spacing, not Whirlpool token mints:
  create.rs#L120, create.rs#L145, create.rs#L152, create.rs#L178
- `deposit_fixed` only checks `token_mint_* == vault.token_mint_*` and position Whirlpool == vault Whirlpool:
  deposit_fixed.rs#L150, deposit_fixed.rs#L156, deposit_fixed.rs#L231
- `withdraw_fixed` forwards stored mints to Orca `collect_fees_v2`:
  withdraw.rs#L410, withdraw.rs#L530, withdraw.rs#L536
- Variable withdrawal is blocked until the fixed position is removed:
  withdraw.rs#L185
- Admin path repeats the same mint assumptions and the same CPI call:
  admin.rs#L150, admin.rs#L308, admin.rs#L314
- Admin close requires prior admin withdrawal steps, so it cannot complete if step 1 fails:
  admin.rs#L736, admin.rs#L737
- Orca CPI wrapper includes both Whirlpool + token mints in `collect_fees_v2` account metas:
  orca.rs#L751, orca.rs#L764, orca.rs#L768

Attack Path

1. Attacker creates vault with real Whirlpool `W` but spoofed mints `X/Y`.
2. Vault initializes normally.
3. Fixed depositor deposits a real position from `W`; checks pass because only `position.whirlpool == vault.whirlpool` and `token_mint_* == vault.token_mint_*`.
4. Variable side deposits and vault starts.
5. At settlement, `withdraw_fixed` calls Orca with `W + X/Y`; Orca rejects mismatch; transaction reverts; position remains in vault.
6. `withdraw_variable` fails with `FixedMustWithdrawFirst` because position still exists.
7. `admin_withdraw_fixed` fails for the same CPI mismatch, so `admin_close_vault` cannot pass step-gating.

Impact

- Permanent liveness failure for affected vaults.
- Fixed side cannot redeem position through protocol path.
- Variable side cannot withdraw earnings due to `FixedMustWithdrawFirst`.
- Admin recovery flow is also blocked.
- Cost to poison is low and locked value can be high.

Recommendation

1. In `create_vault`, derive token mints from Whirlpool account data on-chain and store those values; do not trust caller-provided mints.
2. Add invariant re-checks before the first fixed deposit and before Orca CPIs: `vault.token_mint_a/b` must match Whirlpool’s actual mints.
3. As defense in depth, also validate that `vault.token_mint_a` and `vault.token_mint_b` match the Whirlpool's mints during `deposit_fixed()`, so that an inconsistently configured vault cannot accept a real position even if the creation-time check is bypassed or omitted.

Finally, consider adding an emergency admin or user recovery path that can return the position NFT without first collecting fees. Without such a path, any mint mismatch that reaches a started vault becomes an irreversible lock of the fixed-side position.

Low findings

[L-01] Admin close cannot complete after early_exit_fixed

Description:

admin_withdraw_fixed
requires vault.has_position() (the position NFT is still in the vault), and sets
admin_fixed_withdrawn = 1.

admin_withdraw_variable
requires admin_fixed_withdrawn != 0. After

early_exit_fixed, the position is already transferred out and

has_position()
returns false, so step 1 always fails, blocking steps 2 and 3.

Impact:

If the fixed side exits early and the variable-side holder never withdraws, the remaining fees, penalty, and vault rent are permanently stuck with no recovery path.

Recommendation:

Add an alternative admin flow for post-early-exit vaults that skips step 1 (since the position is already returned) and directly handles variable-side settlement.

[L-02] Acceptance of unverified pools in Saffron from Whirlpool program

Description

`programs/saffron-vault/src/instructions/vault/create.rs` treats any account owned by the official Orca Whirlpool program as an acceptable pool target, but it does not verify which `WhirlpoolsConfig` the pool belongs to, whether the pool comes from a vetted configuration set, or whether the pool satisfies any minimum quality threshold such as existing liquidity or market maturity.

This matters because the Whirlpool program is not a single immutable pool instance. Under the official Whirlpool program, many pool accounts can exist. While creating a new `WhirlpoolsConfig` is admin-gated in Orca, standard pool creation under an existing config

and existing fee tier is permissionless. In other words, a vault creator can deploy a new, real Whirlpool pool under the official Orca program using an already available config/fee-tier combination, and Saffron will treat that pool the same as a long-established pool as long as:

- The account is owned by the official Whirlpool program, and
- The pool exposes a valid `tick_spacing`

The protocol therefore over-trusts Whirlpool program ownership as a proxy for pool quality or ecosystem approval. A malicious vault creator can route users into arbitrary newly created low-fee or low-liquidity pools under the official Whirlpool program without any configuration-level allowlist or explicit disclosure enforced on chain.

This does not appear to be a direct fund-loss bug by itself. The Whirlpool is still a genuine Orca pool, and users can in principle inspect the chosen pool off-chain. However, it widens the trust boundary beyond what many integrators or depositors may reasonably assume from “official Orca Whirlpool” validation alone. If the intended product model is to support only vetted or established pools, the current on-chain checks do not enforce that assumption.

Consider binding vault creation to an allowlisted set of Whirlpool configurations or explicitly approved pools. If full permissionless pool selection is intended, document that trust model clearly and ensure frontends surface the selected Whirlpool, its configuration, fee tier, and basic pool quality indicators prominently before users deposit.

[L-03] `initialize_factory` allows unauthorized address to gain authority

Description

`initialize_factory` in `programs/saffron-vault/src/instructions/factory/initialize.rs` creates the singleton Factory PDA using a fixed seed `[seeds::FACTORY]` with the `init` constraint. The `authority` field is set to whichever `Signer` calls the instruction first:

```
pub struct InitializeFactory<'info> {
    #[account(
        init,
        payer = payer,
        space = Factory::SPACE,
        seeds = [seeds::FACTORY],
        bump,
    )]
    pub factory: AccountLoader<'info, Factory>,

    /// Authority that will own the factory
    pub authority: Signer<'info>,  // ← any signer accepted

    #[account(mut)]
```

```
    pub payer: Signer<'info>,
    // ...
}
```

The handler then stores this caller-chosen authority as the permanent protocol administrator:

```
factory.authority = ctx.accounts.authority.key();
```

There is no restriction binding the initial authority to the program's deployer, upgrade authority, or any predefined governance address. The `init` PDA constraint ensures the instruction can only succeed once, but it does not restrict **who** can call it.

If there is any gap between program deployment and factory initialization — even a single slot — an attacker can monitor the deployment transaction, front-run the initialization, and set themselves as the factory authority. As factory authority, they would control:

- Protocol fee basis points (`set_fee_bps`)
- Protocol fee receiver address (`set_fee_receiver`)
- Default deposit tolerance (`set_default_deposit_tolerance_bps`)
- Admin vault close capability (`admin_withdraw_fixed` , `admin_withdraw_variable` , `admin_close_vault`)
- Authority transfer (`transfer_authority`)

Recommendations

Anchor the initialization authority to a trusted root. The most robust approach is to verify the program's upgrade authority from the `ProgramData` account during `initialize_factory` :

```
    constraint = program_data.upgrade_authority_address == Some(authority.key())
        @ SaffronError::Unauthorized,
)]
pub program_data: Account<'info, ProgramData>,
```

Alternatively, hardcode the expected initial authority pubkey as a constant, or ensure deployment and initialization are executed atomically in the same transaction to eliminate the front-running window.

[L-04] Frozen depositor ATA permanently blocks admin vault close

Description

If any of the three vault token mints (`token_mint_a` , `token_mint_b` , `variable_asset`) have a freeze authority, and that authority freezes a depositor's associated token account, the admin close path is permanently blocked with no recovery mechanism.

`admin_withdraw_variable` transfers earnings directly to the stored `variable_depositor`'s deterministic ATAs. If any destination ATA is frozen by the mint's freeze authority, the transfer CPI fails and the entire transaction reverts. Since `admin_close_vault` requires `admin_withdraw_variable` to complete first, the vault can never be closed; all remaining earnings and rent are permanently locked.

For example, if `token_a` is USDC and Circle freezes the variable depositor's USDC ATA, the transfer of `earnings_to_variable_a` to `variable_depositor_token_a_ata` reverts. This blocks `admin_withdraw_variable` entirely, even though `token_b` earnings and penalty earnings are unrelated to the frozen account.

The admin path has no such fallback because the destination addresses are hardcoded from the vault state and cannot be redirected.

Recommendations:

Consider allowing the admin to pass an alternative destination address in `admin_withdraw_variable`, so funds can be redirected when the depositor's ATA is frozen, or acknowledging this behavior.

[L-05] Missing anchor discriminator validation on manually parsed Orca account

Description

In `create_vault` and `deposit_fixed`, the program manually parses raw account data from Orca Whirlpool and Position accounts. Both paths validate account ownership (`owner == whirlpool_program`) but do not check the 8-byte Anchor discriminator before parsing fields.

The Orca program owns multiple account types (Whirlpool, Position, TickArray, FeeTier, PositionBundle), so an account of the wrong type would pass the owner check while having different data at the parsed offsets.

Exploitation is impractical because subsequent field-level checks (two Pubkey matches + NFT ownership in `deposit_fixed`, tick validation in `create_vault`) make it infeasible to construct a passing fake, but the validation is incomplete compared to `admin_withdraw_fixed`, which correctly validates the Position PDA via seeds constraint.

Recommendations:

Consider adding discriminator validation before parsing, or adding a PDA seeds constraint that matches the pattern in `admin_withdraw_fixed`.

[L-06] `VaultState::get_state()` off-by-one at `end_time` boundary

Description

`Vault::get_state()` uses `current_time <= self.end_time` to return `VaultState::Started`, while all withdrawal and admin instructions use `clock.unix_timestamp >= vault.end_time` to allow post-end actions.

At exactly `end_time`, the helper returns `Started`, but the program already allows withdrawals.

Although `get_state()` is not used for on-chain access control (all handlers check vault fields directly), off-chain indexers and clients that rely on this helper to determine vault state could display an incorrect state at the exact `end_time` boundary moment.

Recommendations:

Consider aligning the helper with instruction logic: use strict inequality so the boundary is treated as ended.

```
if current_time < self.end_time {
    return VaultState::Started;
}
```

[L-07] `admin_close_vault` does not close vault ATAs rent locked

Description

In `admin_close_vault` (step 3 of admin close), the handler sweeps any remaining tokens from the vault's three ATAs (`vault_token_a_ata`, `vault_token_b_ata`, `vault_variable_ata`) to the fee receiver, then closes the vault account (`close = fee_receiver`). The three ATAs are not closed. After the sweep, they have a zero balance but still hold rent-exempt lamports; their authority is the vault PDA.

Each vault-owned ATA holds approximately 0.002 SOL in rent-exempt lamports; with 3–4 ATAs per vault, this amounts to roughly 0.006–0.008 SOL permanently stranded per vault lifecycle.

Once the vault account is closed, that authority no longer exists, so the ATAs cannot be closed later, and their rent is permanently locked.

Recommendations:

Before closing the vault account, consider closing the three vault ATAs with the destination being the fee receiver.

[L-08] Orca incentive rewards are not attributed

Description

The program collects only trading fees (token A/B) via `collect_fees_v2` and does not call `collect_reward_v2`. Orca Whirlpools can have up to three incentive reward tokens per pool; those rewards are never collected by the vault and remain in the position. When the position is returned to the fixed side at `withdraw_fixed` (or `early_exit`), the fixed side can later claim those reward tokens from Orca.

Recommendations:

Consider documenting explicitly that only trading fees (pool token A/B) are collected and attributed to the variable side, and that Orca incentive reward emissions (if any) are not collected by the vault and accrue to the position.

[L-09] Withdraw handlers do not check for admin-withdrawn flags

Description

`withdraw_fixed` If the admin has already run `admin_withdraw_fixed`, the vault has no position (`has_position()` is false) and the vault state is in the admin-close flow. A user calling `withdraw_fixed` will eventually fail on `require!(vault.has_position(), ...)`.

`withdraw_variable` If the admin has already run `admin_withdraw_variable`, the vault's earnings and penalty are zero, and there is no check on `admin_variable_withdrawn`. The instruction does not fail: it succeeds, the user's share is 0, and the user pays for the transaction while receiving nothing; it only allows the user to burn the variable bearer token.

Recommendations:

Consider either failing as early as possible, by throwing a dedicated error, like `AdminAlreadyWithdrawnFixed`, or allowing both to run even after the admin withdrawal to allow users to burn their bearer tokens.

[L-10] Claim leaves user `claim` token ATA open after burn rent not returned

Description

The `claim` instruction burns the user's claim token from `user_claim_token_ata`:

```
/// User's claim token account (source, regular SPL Token)

    mut,
    constraint = user_claim_token_ata.owner == user.key(),
    constraint = user_claim_token_ata.mint == claim_token_mint.key(),
```

```
    constraint = user_claim_token_ata.amount > 0 @ SaffronError::NoClaimTokens,
)]
pub user_claim_token_ata: Box<Account<'info, TokenTokenAccount>>,
```

After the burn, that ATA has a zero token balance but remains open. The rent-exempt lamports used to create the ATA stay locked in the account. The user does not receive them back unless they close the ATA in a separate transaction.

Recommendations:

After burning the claim tokens, close `user_claim_token_ata` with the destination set to `user` so the account's lamports are transferred to the user.

[L-11] Truncation of `remaining_premium` undercharges early exit penalty

Description

The early_exit instruction computes the penalty paid by the fixed side (early exiter) to the variable side. The intended formulas are:

- `remaining_ratio = (end_time - now) / duration`
- `remaining_premium = variable_side_capacity * remaining_ratio`
- `penalty_payment = remaining_premium * (10000 + penalty_bps) / 10000`

Both `remaining_premium` and `penalty_payment` use integer division. Division in Rust truncates toward zero, so the result is rounded down.

That undercharges the early exiter and thus underpays the variable side relative to the exact proportional amount. The variable side is the intended beneficiary of the penalty (they receive the penalty into the vault's variable ATA).

Recommendations:

Consider rounding up when computing `remaining_premium` and when computing `penalty_payment`, in favor of the variable side.

[L-12] Tick bounds in Saffron do not match Orca's canonical range

Description

Saffron uses `MIN_TICK = -887272` and `MAX_TICK = 887272` in `programs/saffron-vault/src/constants.rs`.

```
pub const MIN_TICK: i32 = -887272;
pub const MAX_TICK: i32 = 887272;
```

Orca's Whirlpool program uses `MIN_TICK_INDEX = -443636` and `MAX_TICK_INDEX = 443636` (`whirlpools/programs/whirlpool/src/state/tick.rs`). Saffron's bounds are twice as wide. `create_vault` validates ticks with `tick_lower >= MIN_TICK && tick_upper <= MAX_TICK` , so it can accept a configured range that Orca would treat as out of bounds.

In practice, `deposit_fixed` overwrites the vault's ticks with the position's `tick_lower_index` / `tick_upper_index` from Orca, which are always within Orca's range, so no incorrect state is sent to Orca.

Recommendations:

Consider using Orca's tick index bounds in Saffron: set `MIN_TICK_INDEX = -443636` and `MAX_TICK_INDEX = 443636` .

[L-13] Variable asset ATA created in `deposit_variable` instead of `initialize_vault`

Description

The vault's Associated Token Account for the variable asset (`vault_variable_ata`), which receives premium from the variable depositor, is created in `deposit_variable` with `init_if_needed` . All other vault-owned token accounts, fixed bearer mint, variable bearer mint, and claim token mint, are created in `initialize_vault` . So the variable-asset ATA is the only vault account created later; its rent is paid by the variable depositor (`payer = user`) instead of the vault creator, who pays for all other vault accounts at initialization time. `initialize_vault` already has `variable_asset_mint` and a payer; it could create the vault's ATA at initialization time with the same mint and authority (vault) by adding the associated token program and the variable asset's token program (TokenInterface).

Recommendations:

Consider creating the variable-asset ATA at initialization time so that the creator pays rent.

[L-14] Front-running on `create_vault`

Description

The `expected_vault_id` check is intended to give callers a predictable PDA address, but it creates a front-running window:

1. An attacker observes `factory.next_vault_id = N` on-chain (public state).
2. A legitimate user broadcasts `create_vault` with `expected_vault_id = N`.
3. The attacker copies the call with the same `expected_vault_id = N` and pays higher priority fees.
4. The attacker's transaction lands first → `factory.next_vault_id` becomes `N + 1`.
5. The victim's transaction fails with `InvalidVault`.

Because `create_vault` is permissionless — `creator` is just a `Signer` field with no factory-level allowlist — any wallet can legitimately call it. The attacker pays only one transaction fee per grief cycle and can repeat it indefinitely.

Recommendations

Remove `expected_vault_id` check logic.

[L-15] Missing ownership verification in `withdraw.rs` allows unauthorized withdrawal

Description

Both withdrawal paths in `programs/saffron-vault/src/instructions/vault/withdraw.rs` rely on the caller-provided bearer token account to authorize redemption, but they do not consistently require that the bearer account is owned by the caller.

In `withdraw_variable`, `user_variable_bearer_ata` has no `owner == user` constraint, and the handler only checks that the account balance is non-zero before using `user` as the burn authority. In SPL Token semantics, a delegate is allowed to burn delegated tokens. As a result, any address that has been approved as a delegate for the variable bearer NFT can call `withdraw_variable`, burn the delegated bearer token, and redirect all earnings and penalty proceeds to its own token accounts.

In `withdraw_fixed`, `user_fixed_bearer_ata` is checked for mint and non-zero balance, but it is also missing `owner == user`. The instruction then transfers the vault-held position NFT to `user_position_token_account`, which is derived for the caller, and burns the fixed bearer token using `user` as authority. Therefore, an approved delegate of the fixed bearer token can call `withdraw_fixed` and receive the underlying Orca position NFT in its own account.

This is a real authorization issue rather than a harmless inconsistency. Transferability of bearer tokens may intentionally transfer vault rights, but token delegation is a broader approval primitive that users may grant for unrelated reasons. Elsewhere in the codebase, stricter ownership checks are already used for similar operations:

- `claim.rs` requires `user_claim_token_ata.owner == user.key()`
- `early_withdraw.rs` requires `user_variable_bearer_ata.owner == user.key()`
- `early_exit.rs` requires `user_fixed_bearer_ata.owner == user.key()`

Because `withdraw.rs` omits the same ownership checks, delegated bearers gain stronger rights than the rest of the protocol appears to intend.

Recommendations

Add explicit ownership checks to both withdrawal account structures:

- In `WithdrawVariable`, require `user_variable_bearer_ata.owner == user.key()` and also validate `user_variable_bearer_ata.mint == variable_bearer_mint.key()`.
- In `WithdrawFixed`, require `user_fixed_bearer_ata.owner == user.key()`.

If delegation is intentionally meant to authorize redemption, then the protocol should document that clearly and apply the same rule consistently across `claim`, `early_withdraw`, `early_exit`, and `withdraw`. In its current form, the code treats delegation inconsistently and unintentionally expands withdrawal authority.

[L-16] Unclaimed premium lost to fixed depositor if `claim` is not called in time

Description

After both sides deposit and the vault starts, the fixed depositor must call `claim()` to receive the premium and mint their fixed bearer token. If the fixed depositor never claims on time, both normal withdrawal paths are permanently blocked:

`withdraw_fixed` requires a fixed bearer token that only exists after claiming:

```
// withdraw.rs:370-373
constraint = user_fixed_bearer_ata.mint == fixed_bearer_mint.key(),
constraint = user_fixed_bearer_ata.amount > 0 @ SaffronError::NoFixedSideTokens,
```

`withdraw_variable` requires the position to be removed first, which only happens via `withdraw_fixed`:

```
// withdraw.rs:188-189
require!(!vault.has_position(), SaffronError::FixedMustWithdrawFirst);
```

The only recovery path is admin close. `admin_withdraw_fixed` succeeds (no bearer token check) and returns the position NFT to `vault.fixed_depositor`. `admin_withdraw_variable` transfers trading fee earnings to the variable depositor.

However, the premium (the full `variable_side_capacity` still sitting in `vault_variable_ata`) is never returned to the variable depositor; it is swept to `fee_receiver` as "dust" in `admin_close_vault`:

```
// admin.rs:858-872
let remaining_variable = ctx.accounts.vault_variable_ata.amount; // entire premium
vault_transfer_checked(
    ...
    ctx.accounts.fee_receiver_variable_ata.to_account_info(), // goes to protocol
    ...
    remaining_variable,
    ...
)?;
```

The fixed depositor lent their position, expecting to receive the premium as compensation. In this scenario, they get their position back via admin close, but lose the entire premium to the protocol because they never claimed it.

Recommendations

In `admin_withdraw_variable`, consider checking whether `claim` was exercised (`claim_token_mint.supply == 0` means it is claimed). If unclaimed, transfer the remaining `vault_variable_ata` balance to `fixed_depositor` as the premium they are owed. If already claimed, proceed as current; any remaining balance is legitimate dust and can be swept to `fee_receiver`.

[L-17] Pre-vault-start fees incorrectly assigned to variable side instead of fixed side

Description

The variable side is meant to receive **all** trading fees that accrue **during** the vault duration (between `start_time` and `end_time`). The fixed side provided the liquidity; any fees that accrued on that position **before** the vault started; from the time the position was opened or liquidity was added until `is_started` is set; should go to the fixed side. The program never separates pre-start from in-duration fees. Every fee collection path adds 100% of collected fees to `vault.earnings_a` and `vault.earnings_b`, which are later paid to the variable side. For example, in `admin_withdraw_fixed` (`programs/saffron-vault/src/instructions/factory/admin.rs`):

```
// 2. Collect fees (for variable side earnings)
collect_fees_v2(...)?;
// ...
vault.earnings_a += fees_collected_a;
vault.earnings_b += fees_collected_b;
```

The same pattern appears in the withdraw_fixed path (`withdraw.rs`) and in `early_exit` : all collected fees are credited to variable-side earnings. The vault “starts” when both sides have deposited, in either `deposit_variable` or `deposit_fixed` . The auto-start block in `deposit_variable.rs` looks like this:

```
if ctx.accounts.claim_token_mint.supply == 1 && new_supply == 1 {
    let mut vault = ctx.accounts.vault.load_mut()?;
    let clock = Clock::get()?;
    vault.is_started = 1;
    vault.end_time = clock.unix_timestamp + vault_duration;

    emit!(VaultStarted { ... });
}
```

The same condition and assignment exist in `deposit_fixed.rs` . At that moment, the code does not call Orca `update_fees_and_rewards` and `collect_fees_v2` to pull pre-start fees out of the position, nor does it send those amounts to the fixed depositor. So pre-start fees stay in the position and are later collected together with post-start fees; all are added to `vault.earnings_*` and paid to the variable side.

The fixed side loses fees that accrued before the vault started; the variable side receives them.

Recommendations

When the vault is started (in both `deposit_variable` and `deposit_fixed` , in the block where `claim_token_mint.supply == 1` and `variable_bearer_mint.supply == 1`), before setting `is_started` and `end_time` : if the vault has a position, CPI to Orca `update_fees_and_rewards` then `collect_fees_v2` , then transfer the collected amounts (delta on vault token A/B ATAs) to the fixed depositor’s token A/B ATAs. Alternatively, snapshot the position's fee growth state at vault start (reading the Orca position's `fee_growth_checkpoint_a/b` at the moment `is_started = 1` is set) and only attribute fee growth delta above that snapshot to `vault.earnings_a/b` . This avoids the need for an immediate fee collection CPI at start time and handles the case where the fixed depositor opens the position long before the vault starts.

[L-18] Overaccounting early-exit penalty leads to insufficient vault balance

Description

Code References

- Penalty transfer uses nominal input amount: early_exit.rs:243
- Stored accounting uses nominal amount, not received delta: early_exit.rs:334
- Variable withdraw trusts `penalty_earnings` as fully spendable: withdraw.rs:269, withdraw.rs:303
- Admin withdraw path also trusts full `penalty_earnings` split: admin.rs:616, admin.rs:645, admin.rs:679
- Deposit path has deflationary protection (but early-exit path does not): deposit_variable.rs:129, deposit_variable.rs:150

Root Cause

`early_exit_fixed` records `vault.penalty_earnings = penalty_payment` after transferring from user to vault, but never measures how much the vault actually received. With Token-2022 transfer fees active, recipient credit can be lower than transfer input. That creates accounting insolvency: recorded penalty is greater than the real vault variable-asset balance.

Attack Path

1. Attacker uses a variable asset mint with mutable Token-2022 transfer fee configuration (fee initially zero so deposit succeeds).
2. Vault runs normally and starts.
3. Attacker activates transfer fee and waits until it becomes effective.
4. Attacker calls `early_exit_fixed`; transfer succeeds from sender side, but vault receives less than `penalty_payment`.
5. Contract stores full `penalty_payment` as `penalty_earnings`.
6. Variable user calls `withdraw_variable`; program attempts to transfer full recorded penalty and fails due to insufficient funds.
7. Admin fallback `admin_withdraw_variable` can fail for the same reason when transferring penalty split, blocking admin close step 2.

Impact

- Variable-side withdrawal DoS / fund lock until external intervention.
- Vault lifecycle deadlock risk (admin close sequence can be blocked).
- Economic integrity break: attacker effectively underpays early exit penalty while state reports full penalty.
- Griefing vector against counterparties in attacker-created or attacker-influenced vaults.

Recommendations

1. In `early_exit_fixed`, account by observed balance delta, not nominal input:
2. Read `vault_variable_ata.amount` before transfer.
3. Execute transfer.
4. Reload ATA and compute `received = after - before`.
5. Store `vault.penalty_earnings = received`.
6. Add strict policy option:
7. Either reject any short-receipt (`received != penalty_payment`) with explicit error, or
8. Accept short-receipt but emit both nominal and received in event for transparency.
9. Add defensive payout reconciliation in withdraw/admin paths:
10. Before variable-asset transfer, cap by spendable vault balance or fail with explicit insolvency error (clearer than downstream token failure).